

SAKUM LANG — THE COMPLETE GUIDE (Bible)

Status: Complete working reference v1. This document explains how to use Sakum Lang end to end: philosophy, keywords, the compiler pipeline, every subsystem (engine, database, SIMD, AI, quantum, crawler, server, security, self-learning), the cross-platform machine-code layer (`libprim` + `platform.inc`), the runtime/capability model, build/test, and how to extend the language.

Companion sources of truth (read alongside this guide): - `SAKUM_LANG.md` — design specification & living doctrine (single source of truth) - `SAKUM_HINGLISH.md` — canonical Hinglish keyword glossary - `docs/SAKUM_RUNTIME_ARCHITECTURE.md` — runtime, capability model, PAL - `docs/SYMBOL_MAP.md` — LaTeX notation → Sakum keyword mapping - `EXTENSIONS.sakdoc` — file-type registry - `spec/*.sak` — 21 formal specs, one per subsystem - `libprim/` — the cross-platform machine-code primitive library (runnable)

Table of Contents

1. [What Sakum Lang Is](#)
2. [The Doctrine \(non-negotiable rules\)](#)
3. [The Keyword System](#)
4. 3.1 [Core language keywords](#)
5. 3.2 [The 148 domain keyword registry](#)
6. 3.3 [Devanagari ↔ Hinglish ↔ meaning table](#)
7. [File Types & the .sak Ecosystem](#)
8. [Your First Program](#)
9. [Language Tour \(with runnable examples\)](#)
10. 6.1 [Variables, control flow, functions](#)
11. 6.2 [Vectors / SIMD \(`vektor` \)](#)
12. 6.3 [The Database engine \(`sanchay` \)](#)
13. 6.4 [Systems engineering \(`tantra` \)](#)
14. 6.5 [The engine: heart / pulse / nerve](#)
15. 6.6 [The AI core \(`prajna` \)](#)
16. 6.7 [Quantum core \(`anu` \)](#)
17. 6.8 [LaTeX → Sakum transpiler](#)
18. 6.9 [The web crawler \(`brahma` \)](#)
19. 6.10 [The trigger server & webhook](#)
20. 6.11 [Networking \(`jaal` \)](#)
21. 6.12 [Advanced: OOP, memory safety, self-learn](#)
22. 6.13 [Security & the creator key \(`sutra` \)](#)
23. [The #what Binary-Hash Query Engine](#)
24. [The Compiler Pipeline](#)
25. 8.1 [Stages: lexer → parser → AST → SIR → back end](#)
26. 8.2 [The SIR \(Sanskrit IR\)](#)
27. 8.3 [Back ends](#)
28. 8.4 [WASM emitter](#)
29. [The Cross-Platform Machine-Code Layer](#)
30. 9.1 [platform.inc — the OS/ISA abstraction](#)
31. 9.2 [libprim — the portable primitive library](#)
32. 9.3 [ABI / calling conventions per ISA](#)
33. 9.4 [The ABI Rosetta gotchas \(Apple Silicon\)](#)
34. [The Runtime & Capability Model](#)
 - 10.1 [Runtime layers](#)
 - 10.2 [Capability Token \(CT\)](#)
 - 10.3 [Module format .sakm](#)
 - 10.4 [Memory model](#)
 - 10.5 [Platform Abstraction Layer \(PAL\)](#)
 - 10.6 [Self-healing engine](#)
35. [The Self-Extending Engine \(`self` \)](#)
36. [Building & Testing](#)
 - 12.1 [Build matrix](#)
 - 12.2 [Testing strategy](#)
 - 12.3 [libprim build & QEMU test](#)
37. [How to Add a Primitive / Module / Keyword](#)
38. [Roadmap](#)

1. What Sakum Lang Is

Sakum Lang ("सकम्" — *together / with intent*) is a **Sanskrit-keyword systems language** whose lowest layer is **real machine code**. It is designed so that:

1. Every capability is emitted as **verified machine code**, not interpreted.
2. A single source builds for **Windows / macOS / Linux / bare metal** across **x86_64, i386, ARM64 (incl. Apple Silicon), ARM32, and RISC-V64**.
3. Scientific notation, vectors/math, security, AI, and a self-learning engine live **in the core** — not as foreign dependencies.
4. It is **quantum-ready**: qubit ops exist in the core (simulator now, real backend later).
5. It uses a **creator-owned hash key** (`sutra`) for encrypted transmission — **no SHA-256** anywhere.
6. It **can only be (re)built by its own code or by machine-level code** — never by a foreign high-level runtime.

Sakum Lang is *not* a high-level language with an LLVM backend. It is a discipline: write the lowest common denominator in hand-tuned assembly primitives, wrap them in a stable C ABI, and let higher layers (keywords, engine, modules) compose them.

2. The Doctrine (non-negotiable rules)

From `SAKUM_LANG.md §2` and `spec/spec_doctrine.sak`. These are hard constraints.

DO: - Emit **raw machine-level code** from back ends (never host-language execution). - Make **every construct addressable by a binary hash** (`#what`). - Use the **creator-owned key** (`sutra`), not SHA-derived. - Treat `vektor` (**SIMD**) as **first-class**.

DON'T: - **No foreign high-level runtime** to (re)build the core. Bootstrap must reach self-hosting from machine-level code only. - **No foreign host language** (Python, bash, Go, Rust, C, ...) *anywhere* — not the core, libraries, engine, kernel, database, or helper tools. Anything missing must be created at machine level (raw x86-64 / ARM64 / RISC-V assembly). Retired and reimplemented in assembly: `serve.py` → `tools/serve.s`, `gen_sir.sh` → assembly, `sakum_tex.py` → `assembly/sakum_tex.s`. - **Don't silently drop security checks** in production. - **Don't erase prior doctrine** — preserve and reconcile. - **Don't leak the creator hash key**; it is installed, never printed in clear. - **Don't hardcode a single ISA**; back ends are pluggable. - **Don't make it slower than necessary**.

3. The Keyword System

Sakum keywords are **Hinglish (romanized Sanskrit) only** — typeable ASCII, no Devanagari required, and **no pure-English keywords** (`let / fn / if / return` are rejected by the lexer). The canonical spelling lives in `SAKUM_HINGLISH.md`. The machine-level registry is `assembly/sakum_keywords.s`; the spec mirror is `spec/spec_keywords.sak`.

3.1 Core language keywords

Hinglish	Meaning	Replaces
<code>aarambh</code>	program / main	आरम्भ
<code>naam</code>	declare variable	नाम / चर
<code>kriya</code>	function	क्रिया / सूत्र
<code>yadi</code>	if	यदि
<code>anyatha</code>	else	अन्यथा
<code>yavat</code>	while loop	यावत् / जबतक
<code>paryantam</code>	counted (for) loop	पर्यन्तम्
<code>vapsa</code>	return	वापस
<code>satya / asatya</code>	true / false	सत्य / असत्य
<code>shunya</code>	nil / none	शून्य
<code>lek</code>	output / print	लेख / मुद्रण
<code>hriday</code>	engine allocator (heart)	हृदय
<code>spand</code>	engine tick (pulse)	स्पन्द
<code>naadi</code>	signal bus (nerve)	नाडी

Hinglish	Meaning	Replaces
sutra	creator key / thread / rule	सूत्र
varg	class (OOP vtable)	वर्ग
vyakhya	explain an error	व्याख्या
svadhyaya	self-learn	स्वाध्याय
santripti	saturating arithmetic	संतृप्ति:
vektor	SIMD vector	वेक्टर
ank	integer vector element type	अङ्क
paripath	quantum circuit	परिपथ
dvar	quantum gate	द्वार
map	measure (qubit)	माप
brahma	web crawler / scraper	ब्रम्ह
pariksha	self-test block	परीक्षा
aur / athava	logical and / or	और / अथवा
lambai	length	लंबाई
sanchay	unified database engine	सञ्चय
kech	key/value store	केच
anukra	vectorless index	अनुक्र
grantha	graph store	ग्रन्थ
tantra	systems engineering	तन्त्र
anvesh	search / lookup	अन्वेष
sarani	hash table	सारणी
chakra	ring buffer	चक्र

3.2 The 148 domain keyword registry

Reserved across the **entire** ecosystem, implemented as machine-level library functions in `assembly/sakum_lib_domains.s` (+ arm64/riscv64 ports) and the registry in `assembly/sakum_keywords.s`. Grouped by subsystem (ids 0–12):

ID	Subsystem	Example keywords
0	TYPES	vastu object, rupa type, akruti struct, samuha collection, gan group, kosh map, shrunkhala list, rekha array, bindu point, jod tuple, prakar variant, lakshan trait
1	FUNC	ahvaan invoke, pravah pipeline, sangrah collect, vibhaj split, milan merge, parivartan transform, anukram sequence, punaravartan recursion, pratinidhi delegate, vistrit expand, sankuchit reduce
2	CONC	prakriya process, samayojan sync, samantar parallel, sandesh message, pravahan stream, vahini channel, prerak sender, grahak receiver, pratiksha await, jagrit wake, nidra sleep
3	MEM	smriti memory, smritikosh cache, aavantan allocate, mukti free, sthaan address, suchak pointer, sandarbh reference, sthir immutable, chal mutable, raksha protection
4	FS	granth file, granthagar directory, path, patan read, lekhan write, jodan append, pratilipi copy, sthanantar move, naamkaran rename, vinash delete
5	NET	jaal network, sampark connect, viyog disconnect, pravesh login, nirgam logout, agrah request, uttar response, prasaran broadcast, grahan receive, prasthaan send, dvaar port, marg route
6	AI	prajna intelligence, buddhi reasoning, chintan inference, smaran recall, adhigam learning, abhyas training, nirnay decision, drishti vision, shravan audio, vak speech, bhasha language, manan reflection, kalpana imagination, chetana awareness, sankalp planning
7	ROBOT	hasta actuator, netra camera, karna mic, charan locomotion, gati movement, disha direction, veg speed, santulan balance, spandan sensor event, sparsh touch
8	QUANT	anu quantum, kan particle, adhithiti superposition, samyojan entanglement, tarang wave, kaksha state, kampan oscillation, urja energy, pariman measurement, nirikshan observe
9	COMPILER	varna token, pad symbol, vakya syntax, artha semantics, vishleshan parser, sankalan compile, nirman build, bandhan link, chalana execute, sudhar optimize, pariksha validate, utpadan generate
10	SEC	raksha secure, gopan encrypt, vigopan decrypt, praman authenticate, adhikar authorize, mudra signature, kunji key, gupt private, sarvajanik public, kavach shield/firewall
11	DIST	

ID	Subsystem	Example keywords
		mandal cluster, ganana compute, vitaran distribute, samanvay coordinate, samvedan synchronize, pratinidhi replica, nayak leader, anuyayi follower, matdaan consensus, sthirata consistency
12	LIVING	hriday Heart, manas Mind, buddhi Reasoning, chetana Conscious context, smriti Long-term memory, sankalp Goal, prerna Motivation, indriya Sensor, prana Runtime lifecycle, atma Root identity

The runtime API exposed by the registry (`spec/spec_keywords.sak`): `naam node` , `naam hash = #what ...` , `lek(query("..."))` , `lek(heartbeat())` , `sakum_kw_count()` , `sakum_kw_lookup(name)` , `sakum_kw_category(idx)` , `sakum_kw_name(idx)` , `sakum_domain_dispatch(id, a, b)` .

3.3 Devanagari ↔ Hinglish ↔ meaning table

A combined reference (from `SAKUM_LANG.md` §5 + `SAKUM_HINGLISH.md`):

Devanagari	Hinglish	Meaning
आरम्भ	aarambh	program / main
नाम	naam	declare variable
क्रिया	kriya	function
यदि	yadi	if
अन्यथा	anyatha	else
यावत्	yavat	while
पर्यन्तम्	paryantam	counted loop
प्रत्यागम	vapsa	return
सत्य / असत्य	satya / asatya	true / false
शून्य	shunya	nil
लेख	lek	output / print
हृदय	hriday	engine allocator (heart)
स्पन्द	spand	engine tick (pulse)
नाडी	naadi	signal bus (nerve)
सूत्र	sutra	creator key / thread / rule
वर्ग	varg	class (vtable)
व्याख्या	vyakhya	explain error
स्वाध्याय	svadhyaya	self-learn
संतृप्तिः	santripiti	saturating arithmetic
वेक्टर	vektor	SIMD vector
अङ्क	ank	integer vector element
परिपथ	paripath	quantum circuit
द्वार	dvar	quantum gate
माप	map	measure qubit
ब्रम्ह	brahma	web crawler
परीक्षा	pariksha	self-test block
और / अथवा	aur / athava	logical and / or
लंबाई	lambai	length
सञ्चय	sanchay	database engine
केच	kech	key/value store
अनुक्र	anukra	vectorless index
ग्रन्थ	grantha	graph store
तन्त्र	tantra	systems engineering
अन्वेष	anvesh	search / lookup
सारणी	sarani	hash table
चक्र	chakra	ring buffer

4. File Types & the .sak Ecosystem

From `EXTENSIONS.sakdoc` — every artifact carries a purpose-specific extension so the compiler/tooling can recognize specialized data directly. Enforced by `tools/sakum_ext.py`; `make ext-check` scans the tree for unknown extensions, `make ext-pdf` regenerates the `.pdf` / `.tex` docs.

Core set:

Ext	Meaning
<code>.sak</code>	Sakum source code
<code>.sakm</code>	Sakum module / package source
<code>.sakh</code>	Header / interface declarations
<code>.sakpkg</code>	Package manifest
<code>.sakproj</code>	Project configuration
<code>.saklock</code>	Dependency lock file
<code>.sakir</code>	Intermediate Representation (SIR)
<code>.sakast</code>	Abstract Syntax Tree
<code>.sakbc</code>	Bytecode
<code>.sakobj</code>	Object file
<code>.saklib</code>	Static library
<code>.sakdll</code> / <code>.sakso</code> / <code>.sakdylib</code>	Dynamic library (Win / Linux / macOS)
<code>.sakexe</code>	Platform-independent executable bundle
<code>.sakdoc</code>	Language documentation
<code>.sakapi</code>	API documentation
<code>.saktest</code>	Unit tests
<code>.sakbench</code>	Benchmarks

Domain data extensions: `.sakmath` (math), `.sakphys` (physics), `.sakchem` (chemistry), `.sakbio` (biology), `.sakquant` (quantum algorithms), `.sakml` (ML models), `.saktensor` (tensors), `.sakproof` (proofs), `.sakgraph` (graphs), `.sakquery` (query scripts), `.sakschema` (schemas), `.sakcfg` (config), `.sakcache` / `.sakidx` / `.sakdb` / `.saklog` (KB / index / DB / logs).

5. Your First Program

examples/hello.sak :

```
naam x = 10;
lek(x);

yadi (x > 5) {
  lek("बृहत्");
} anyatha {
  lek("लघु");
}

yavat (x > 0) {
  lek(x);
  x = x - 1;
}

kriya योग(a, b) {
  vapsa a + b;
}
lek(योग(3, 4));

naam v = vec(1, 2, 3);
lek(latex(v));

naam q = qubit(1);
lek(measure(q));

lek(query("learn from mistake in memory"));
```

```
lek(sutra_key());
lek(heartbeat());
```

Compile path (assembly front end is the bootstrap — see §8):

```
gcc -arch x86_64 assembly/sakum_eval.s -o /tmp/eval
# ... feeds examples/hello.sak through lexer → parser → evaluator
```

The same source is portable: lower it to ARM64 / RISC-V via the SIR back ends and the `platform.inc` macros — no source changes.

6. Language Tour (with runnable examples)

All examples follow the keyword surface in §3 and are addressable by `#what`.

6.1 Variables, control flow, functions

```
naam counter = 0;
yavat (counter < 5) {
  yadi (counter % 2 == 0) {
    lek("even");
  } anyatha {
    lek("odd");
  }
  counter = counter + 1;
}

kriya factorial(n) {
  yadi (n <= 1) { vapsa 1; }
  vapsa n * factorial(n - 1);    # punaravartan (recursion)
}

pariksha {
  lek("5! = " factorial(5));
}
```

6.2 Vectors / SIMD (vektor)

`vektor` is first-class (§1.11). The compiler emits AVX2 / AVX-512 (x86-64), NEON (ARM64), or RVV (RISC-V) — no intrinsics needed.

```
vektor A = vec(1, 2, 3, 4, 0, 0, 0, 0);
vektor B = vec(5, 6, 7, 8, 0, 0, 0, 0);
vektor C = A + B;                # 6 8 10 12 — one vpaddd on x86-64
naam n = lambai(C);              # length
lek(simd_info().isa);            # "avx2" / "neon" / "rvv"
lek(simd_info().lanes);          # 8 / 4 / 8 ...
```

The canonical machine-level demo is `assembly/sakum_simd.s` (AVX2, 8 lanes):

```
vmovdqu ymm0, [rip + A]
vmovdqu ymm1, [rip + B]
vpaddq ymm2, ymm0, ymm1          ; C = A + B in one instruction
vmovdqu [rip + C], ymm2
```

Build/run: `gcc -arch x86_64 assembly/sakum_simd.s -o /tmp/simd && /tmp/simd`.

6.3 The Database engine (sanchay)

A single machine-level store (`assembly/sakum_db.s`) unifying **six** primitive data shapes, all addressable by `#what` and portable across ISAs/OSes. All six share the `hriday` allocator + `sutra` crypto + `naadi` bus.

Hinglish	Shape	Notes
<code>kech</code>	key/value	Redis/Valkey style, sutra-encrypted at rest
<code>vektor</code>	vector (ANN)	SIMD L2/cosine distance, stateful index
<code>anukra</code>	vectorless	B-tree / inverted classical index
<code>sthit</code>	stateful	mutable, persisted, hash-addressable (durable memory)

Hinglish	Shape	Notes
asthit	stateless	pure key→value, no persistence, no mutation
grantha	graph	typed edges + naadi -driven traversal

```
kech_put("user:1", "Amit");
naam v = kech_get("user:1");

vektor Q = vec(0.1, 0.2, 0.3, 0, 0, 0, 0, 0);
vektor_insert(1, Q);
vektor_search(Q, 5);           # ANN over SIMD

grantha_node(1, "root");
grantha_edge(1, 2, "child");
grantha_traverse(1, 3);

sthit_put("memory:1", 100);    # survives across pulses
asthit_put("req:1", 200);      # request-scoped, no mutation
```

6.4 Systems engineering (tantra)

Algorithm + data structure + system design written **once** in Sakum, lowered to every target. Demonstrated in assembly/sakum_sys.s (+ _arm64 , _riscv64).

```
# binary search (anvesh)
kriya binary_search(arr, n, key) {
    naam lo = 0; naam hi = n - 1;
    yavat lo <= hi {
        naam mid = (lo + hi) / 2;
        yadi arr[mid] == key { vapsa mid; }
        anyatha yadi arr[mid] < key { lo = mid + 1; }
        anyatha { hi = mid - 1; }
    }
    vapsa -1;
}

# open-addressing hash table (sarani)
kriya hash_put(table, m, k, v) {
    naam h = k % m; naam i = 0;
    yavat i < m {
        naam slot = (h + i) % m;
        yadi table[slot].key == 0 {
            table[slot].key = k; table[slot].val = v; vapsa 1;
        }
        i = i + 1;
    }
    vapsa 0;           # full
}

# ring buffer (chakra) drained on every pulse (spand)
kriya ring_produce(rb, cap, item) { lek(query("nerve emit " item)); vapsa 1; }
```

The cross-ISA matrix: one algorithm × {x86-64, ARM64, RISC-V} × {macOS, Linux, Windows} = 18 target builds, all sharing hriday + sutra + naadi .

6.5 The engine: heart / pulse / nerve

- hriday (हृदय) — runtime allocator + GC heartbeat; fixed pool with used-flag and 0xCAFE canary; double-free → -1 .
- spand (स्पन्द) — periodic tick driving schedulers and the learning loop.
- naadi (नाडी) — event/signal bus connecting components; producer/consumer queues (chakra) drain on each pulse.

```
hriday.alloc(pool, i);           # allocate from the heart pool
spand.tick();                     # drive one pulse
naadi.emit("webhook.update");    # fire a nerve signal
```

6.6 The AI core (prajna)

Implemented in assembly/sakum_ai.s + spec spec/spec_ai.sak . Lifecycle (brahma.learn folds each Knowledge chunk into a weight matrix W via a custom FNV-1a-style fold; forward pass Vout = W · Vin (8×8); a self-update ledger is appended):

```
# walk Knowledge/ (bounded 3-level), ingest #what hex + node name + query into W
sutra ai_lifecycle() {
    naam walk_dir = 1; naam ingest = 1; naam forward = 1; naam self_update = 1;
    vapsa walk_dir + ingest + forward + self_update;
}
# RAM-aware neuron scaling: +8 per chunk, cap 64
sutra neurons_loaded(chunks) {
    naam n = chunks * 8; yadi (n > 64) { n = 64; } vapsa n;
}
```

The `Knowledge/` tree (85 manifest nodes: 8 depth-1 + 77 depth-2) is the corpus; `spec/spec_datamodel.sak` documents the manifest format and the self-update ledger tick.

6.7 Quantum core (`anu`)

`spec/spec_keywords.sak` + `spec/spec_quantum.sak` (planned emitter `assembly/sakum_quantum.s`, format **QCB1**):

```
kriya dom_superposition(a, b) { vapsa a | b; }      # adhithiti
kriya dom_entangle(a, b)    { vapsa a ^ b; }        # samyojan
kriya dom_encrypt(a, key)   { vapsa a ^ key; }      # gopan (sutra xor)

paripath c = circuit(3);                                # build a 3-qubit circuit
dvar(c, "H", 0);                                         # Hadamard on q0
dvar(c, "CX", 0, 1);                                    # CNOT q0->q1
naam bits = map(c);                                     # measure with collapse
```

Circuits serialize to a portable binary (`circuit_binary(c)`) and a LaTeX unitary (`circuit_formula(c)`). Simulator now; real backend later.

6.8 LaTeX → Sakum transpiler

`assembly/sakum_tex.s` (machine-level, no host language). Reads LaTeX math from stdin, emits Sakum to stdout. Grammar: `expr → term (('+'|'-') term)*`; `term → ['-'] factor (('*'|'/') factor)*`; `factor → atom ('^' factor)? | atom ('_' atom)? | atom implicit-mult`.

LaTeX	Sakum
<code>\frac{a}{b}</code>	<code>(a / b)</code>
<code>\sqrt{x}</code>	<code>sqrt(x)</code>
<code>\sqrt[n]{x}</code>	<code>root(n, x)</code>
<code>x^y</code>	<code>pow(x, y)</code>
<code>x_y</code>	<code>idx(x, y)</code>
<code>4ac</code>	<code>4 * a * c</code> (implicit mult)
<code>\begin{bmatrix}...</code>	<code>mat(...)</code>

```
printf '%s' '\frac{-b+\sqrt{b^2-4ac}}{2a}' | \
gcc -arch x86_64 -include assembly/platform.inc assembly/sakum_tex.s -o /tmp/tex && /tmp/tex
# -> naam tex_expr = (-b + sqrt(bpow(2) - 4 * ac) / 2 * a)
```

6.9 The web crawler (`brahma`)

`assembly/sakum_bramann.s` — a from-scratch x86-64 HTTP/1.1 GET client, an HTML scraper (extracts `<title>` and `<a href>` with its own loop, no regex lib), and a **quantum-learn** loop that folds each sphere into a binary hash and records research in `research.md`.

`assembly/sakum_webhook.s` is a raw-assembly socket/bind/listen/ accept receiver that parses `POST /update`, emits a `webhook.update` nerve signal, and runs a self-update cycle.

```
brahma.crawl("https://example.com");      # raw HTTP GET
brahma.scrape_title(buf, len);             # extract <title>
brahma.learn("tools/sakum_bot.sh --once"); # fold into ledger
```

6.10 The trigger server & webhook

`tools/serve.s` (replaces retired `serve.py`) — a native nerve-bus hub with routes: `POST /update` (fork + `sakum_bot.sh`, bump `webhook.update`), `GET /status` (dump `memory.md`), `GET /nerve` (subscriber counts), `GET /index` (serve HTML), `404`. Triggered by timer pulse, webhook, or poll — all runnable locally via `tools/com.sakum.bot.plist` (launchd) or `serve.sh`.

6.11 Networking (`jaal`)

`spec/spec_net.sak` — port scanner (`socket` + per-port `connect` , prints `[OPEN]` / `[closed]`) and a BPF sniffer (`/dev/bpf0` , `BIOCSETIF` , decode `Eth`→`IPv4`→`TCP/UDP`). Both raw-syscall, portable via PAL.

```
port_scan("127.0.0.1", 1, 1024);
packet_sniff("en0", 50);
```

6.12 Advanced: OOP, memory safety, self-learn

From `spec/spec_adv.sak` + `assembly/sakum_adv.s` :

```
# varg: runtime-built vtable, object buffer 8-byte aligned (.balign 8)
varg Shape {
    sutra area()    { vapsa 0; }
    sutra describe(){ vapsa 0; }
}

# hriday: fixed pool, 0xCAFE canary; double free -> -1
kriya heart_free(pool, i, used) {
    yadi (used == 0) { vapsa -1; }    # already freed
    vapsa 0;
}

# vyakhya: error code -> human message (codes 1..5)
kriya vyakhya(code) {
    yadi (code >= 1 aur code <= 5) { vapsa code; }
    vapsa 0;
}

# svadhyaya: record mistake, return patch note
kriya svadhyaya(mistake) { brahma.learn(mistake); vapsa 1; }
```

6.13 Security & the creator key (`sutra`)

- Creator-owned key installed per system (env `SAKUM_SUTRA_KEY` or file `sakum_key.txt`) — **not** SHA-derived. No SHA-256 anywhere.
- Encrypted transmission over the comms layer using the installed key (`gopan` / `vigopan`). Post-quantum-safe primitives where available.
- `assembly/sakum_lib_crypto.s` provides the double-FNV-1a 64-bit digest (no SHA) used for the `#what` folds.
- The key is never printed in clear; `sutra_key()` returns only a presence signal.

7. The `#what` Binary-Hash Query Engine

Every construct is addressable by a **binary hash** (`#what`) — a literal 64-nibble (256-bit) hexadecimal key written in source, e.g.:

```
naam node = "Sakum AI Core";
naam hash = #what f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8fe3;
lek(query("load Sakum AI Core"));
lek(heartbeat());
```

- **No SHA-256.** The corpus is folded with **FNV-1a**; the assembler emits the `#what` key verbatim as data and the runtime walker resolves it by hex equality.
- Queries are categorized by type and split into per-observation files under `query_logs/`.
- `#` notes carry the engine's suggestions in binary/machine form. The `Knowledge/` tree, trusted PL sources, and webhook targets all become `#what` notes in the ledger.

`docs/SYMBOL_MAP.md` is the authoritative LaTeX-notation → keyword map (the PDF's $\partial \nabla \Sigma \int \langle \cdot \rangle \times$ symbols are *human-facing descriptions* of the Sakum keywords; they never reach the assembler).

8. The Compiler Pipeline

From `SAKUM_LANG.md` §3 + `spec/spec_pipeline.sak`. All stages are hand-written assembly in `assembly/` (the bootstrap); the goal is self-hosting (Sakum compiling Sakum).

8.1 Stages: lexer → parser → AST → SIR → back end

```
source.sak → lexer (asm) → parser (asm) → AST → SIR → { wasm | x64/ARM asm | SIMD }
```

and in parallel the `self` engine grows a code buffer between `spand` (pulse) ticks.

Worked constant-fold example (`spec_pipeline.sak`): `let x = 2 + 3*4; let y = x*x; y - 10; →`

```
lex → parse → ir_emit → fold → codegen    # 5 stages
x = 2 + 12 = 14 ; 14*14 = 196 ; 196 - 10 = 186
```

`assembly/sakum_eval.s` is the self-hosted front end: hand-written lexer + recursive-descent parser + evaluator.

8.2 The SIR (Sanskrit IR)

The portable hub the back ends lower from. Example (`spec/spec_sys.sak`):

```
stha r0 0          # lo = 0
stha r1 n          # hi = n-1
.loop: chot r1 r0   # while lo <= hi
  yoj  r2 r0 r1    # mid = (lo+hi)
  ...
```

Back ends: **Native** (lowers SIR → x86/ARM/RVV), **VM** (runs `sirvm`), **AI** (explains the IR).

8.3 Back ends

- **x86-64**: AVX2/AVX-512 SIMD.
- **ARM64**: NEON.
- **RISC-V rv64**: RVV.
- **WASM**: byte-by-byte emitter (see §8.4).
- **Self-growing code buffer** (`assembly/sakum_self.s`): appends generated instruction bytes (continuous growth) — the autonomy path.

8.4 WASM emitter

`assembly/sakum_wasm.s` emits a spec-valid `.wasm` binary byte-by-byte (LEB128), verified by `wasm-validate` / `wasmtime` / `node`.

```
# func run() -> i32 { 1 + 2*3 } => 7
sutra wasm_result() { vapsa 1 + 2 * 3; } # 7
```

9. The Cross-Platform Machine-Code Layer

This is the foundation every higher layer builds on. Two pieces: `platform.inc` (the OS/ISA abstraction) and `libprim` (the portable primitive library).

9.1 `platform.inc` — the OS/ISA abstraction

Every arch `.s` starts with `#include "platform.inc"`. It defines: - `CDECL(name)` → on macOS prepends `_` (`_prim_memcpy`), elsewhere bare. The *only* place OS symbol differences live. - `TEXT_SECTION` → `.section __TEXT,__text` (macOS) or `.text` (Linux/Windows). - OS detection from `-DMACOS` / `-DLINUX` / `-DWINDOWS` (set by the Makefile).

```
#ifdef PLAT_MACOS
#define CDECL(name) _##name
#else
#define CDECL(name) name
#endif
```

9.2 `libprim` — the portable primitive library

`libprim/` is the **first fully working, test-covered** piece of Sakum Lang and the template every module should follow. One C header, one `.s` per arch, a Makefile that builds a static lib per target, and a self-check test.

API (`libprim/include/prim.h`):

```

void *prim_memcpy (void *dst, const void *src, unsigned long n);
void *prim_memset (void *dst, int c, unsigned long n);
size_t prim_strlen (const char *s);
int prim_memcmp (const void *a, const void *b, unsigned long n);
int prim_sadd_overflow(long a, long b, long *r);
int prim_uadd_overflow(unsigned long a, unsigned long b, unsigned long *r);
int prim_smul_overflow(long a, long b, long *r);
int prim_umul_overflow(unsigned long a, unsigned long b, unsigned long *r);
long prim_sadd_sat(long a, long b);
long prim_smul_sat(long a, long b);
double prim_fsqrt(double x);
double prim_fma (double a, double b, double c);
double prim_fabs (double x);

```

Layout: include/prim.h , platform.inc , src/{x86_64,i386,arm64,arm32,riscv64}/prim.s , tests/test_prim.c , Makefile , README.md .

Build & test:

```

make test # host test (macOS arm64 / x86_64)
make OS=macos ARCH=arm64 lib # one static lib per target
make OS=macos ARCH=x86_64 lib
make OS=linux ARCH=riscv64 lib
make OS=windows ARCH=arm64 lib
make lipo # macOS fat lib = arm64 + x86_64
make OS=linux ARCH=riscv64 qtest # QEMU user-mode cross run
make qtest-all # arm64 + arm32 + riscv64
make check-symbols # assemble every arch, assert 13/13 symbols

```

Status: x86_64 & arm64 — 19/19 tests pass (native). i386 / arm32 / riscv64 — assemble cleanly, all 13/13 symbols exported (full execution needs QEMU user-mode, not shipped on macOS).

9.3 ABI / calling conventions per ISA

From spec/spec_abi.sak :

Class	x86_64	arm64	riscv64
Int args	rdi rsi rdx rcx r8 r9	x0-x7	a0-a7
Int return	rax	x0	a0
Float return	xmm0	d0	fa0
Callee-saved	rbx rbp r12 r13 r14 r15	x19-x28	s0-s11
Stack align	16 at every call	16	16

i386 (cdecl): args pushed right-to-left, result `eax` , double in `st(0)` . arm32 (AAPCS + VFP): args `r0-r3` , return `r0 / d0` , needs `-mfpu=vfpv3-d16` .

Gotcha: the one-operand `imul r/m` and `mul r/m` write the **full 128-bit result into** `rdx:rax` , clobbering `rdx` . If `rdx` holds your output pointer, save it first (`mov rcx, rdx`) — exactly what `prim_smul_overflow / prim_umul_overflow` do.

9.4 The ABI Rosetta gotchas (Apple Silicon)

From spec/spec_abi.sak — critical when running x86-64 under Rosetta on Apple Silicon: - **No raw syscalls** → `SIGSYS` ; use `libc` `_open$NOCANCEL` etc. - `_fstat$INODE64` **verify** — `_open` can return garbage deep in the stack. - **No** `printf` + `fflush` **in walk loops** — can hang; use `_write` to fd 2. - **Save** `r8 / r9` in ingest loops (scratch, not callee-saved).

10. The Runtime & Capability Model

From docs/SAKUM_RUNTIME_ARCHITECTURE.md + core/sakum_runtime.s .

10.1 Runtime layers

```

AGENT      - Goal Planner, Policy Engine, Capability Broker
CAPABILITY - Registry, Tokens, Dependency Graph, Scheduler
MODULE     - Format, Loader, Verifier, Versioning

```

```
PLATFORM    - Syscall ABI, Memory, Time, Entropy, Net    (PAL)
HARDWARE    - CPU Rings, MMU, NX, MPK/PKR, TEE, RNG
```

Core principles: **Zero Trust** (signed, verified, token-gated), **No Runtime** (static linking only; no loader/GC/interpreter), **Deterministic** (fixed layout, bounded execution, no runtime allocation), **Portable** (one source → all targets), **Auditable** (every syscall logged), **Self-Healing**.

10.2 Capability Token (CT)

```
struct CapabilityToken {
    uint64_t  capability_id;    // SHA-256(name) truncated
    uint64_t  module_hash;     // SHA-256(module)
    uint32_t  version_major, version_minor, version_patch;
    uint64_t  issued_timestamp, expires_timestamp; // 0 = never
    uint32_t  permissions_bitmap; // READ=1 WRITE=2 EXEC=4 NET=8 FS=16 IPC=32
    uint64_t  parent_token_id;  // delegation chains
    uint8_t   signature[64];    // Ed25519 by CA
};
```

Declared statically in `capability.def`; looked up $O(1)$ at runtime; revocation via bitmap + generation counter.

10.3 Module format `.sakm`

```
MAGIC "SAKM" | VERSION 2 | ARCH (0=x86_64 1=ARM64 2=RISC-V64) | FLAGS
HEADER CRC32
METADATA: name, version, capability_id, deps[], entry/init/fini/health offsets,
          required_perms, max_stack, max_heap, max_cycles
CODE SECTION (raw machine code)
RODATA SECTION
RELOCATION TABLE (optional, for ASLR)
SYMBOL TABLE (optional, debug)
SIGNATURE (Ed25519, 64 bytes)
```

Flags: `REQUIRES_STACK_CANARY`, `REQUIRES_ASLR`, `REQUIRES_W_XOR_X`, `HAS_INIT/FINI/ HEALTH_FN`, `ALLOWS_REENTRANT`, `IS_PURE`.

10.4 Memory model

Static layout: `.text` (RX) · `.rodata` (R) · `.data` (RW) · `.bss` (RW) · STACK POOL (64 KB × workers, guard pages) · HEAP POOL (fixed arena) · MODULE SLOTS · LOG RING (append-only audit). **No heap allocation at runtime** — all pools fixed at compile time.

10.5 Platform Abstraction Layer (PAL)

One syscall surface for every OS/arch (`platform/macos_x86_64.s`, `platform/linux_x86_64.s`, `assembly/sakum_sys*.s`). Single entry `pal_syscall(n, args[6])`. Syscall set: `SYS_READ` `WRITE` `OPEN` `CLOSE` `MMAP` `MUNMAP` `MPROTECT` `SOCKET` `BIND` `LISTEN` `ACCEPT` `CONNECT` `SEND` `RECV` `TIME` `NANOTIME` `RANDOM` `YIELD` `EXIT`. `core/sakum_runtime.s` wires `_pal_*` into the scheduler, health monitor, audit log, and capability dispatch.

10.6 Self-healing engine

Per-module `HealthMetrics` (invocations, cycles, mem peak, success_rate EWMA, health_score). Policies: `health_score<128` → degrade; `success_rate<0.95` → restart; `latency_p99>2×budget` → throttle; `mem_peak>90%` → compact; `crash>3/min` → quarantine; `signature_mismatch` → reject. Root-cause isolation walks the reverse dependency graph and simulates counterfactuals ("if B healthy, would A fail?").

11. The Self-Extending Engine (`self`)

From `spec/spec_self.sak` + `assembly/sakum_self.s`: a code buffer that grows by appending generated instruction bytes — continuous growth between pulses.

```
# emit mov eax,7 (B8 07 00 00 00) + add eax,35 (83 C0 23) => 8 bytes
sutra self_program() {
    naam len = 0;
    len = grow(0, len, 0xB8); len = grow(0, len, 0x07); len = grow(0, len, 0x00);
    len = grow(0, len, 0x00); len = grow(0, len, 0x00);
    len = grow(0, len, 0x83); len = grow(0, len, 0xC0); len = grow(0, len, 0x23);
```

```
vapsa len;                                # 8
}
```

`self.create(...)` / `self.update(...)` patch the language and its modules live. The self-updater bot (`tools/sakum_bot.sh` , driven by `tools/serve.s`) reads `learn.md` / `memory.md` , webfetch-checks PL updates, writes patches to `self/patches/` , recompiles the `assembly/` core, and rolls back on any compile failure.

12. Building & Testing

12.1 Build matrix

From `spec/spec_build.sak` — every core assembles with `gcc -arch x86_64 ...` and is doctrine-compliant:

```
pipeline simd ai tracker serve scan sniff wasm self
```

Plus the cross-ISA ports (`_arm64` , `_riscv64`) for `sys/keywords/libs/tracker`.

12.2 Testing strategy

Level	Method
Unit	Per-module assembly tests (<code>test_*.s</code> / <code>libprim/tests/test_prim.c</code>)
Integration	Capability-graph fuzzing (random valid subgraphs)
Chaos	Inject delay / OOM / crash / corrupt-return
Formal	Model-check scheduler + healer (TLA+)
Perf	Cycle-accurate sim (gem5) + real hardware

12.3 libprim build & QEMU test

```
make test                # host (macOS arm64/x86_64): 19/19
make check-symbols       # fast CI gate: assembles all 5 arches, 13/13 symbols
make qtest-all           # cross via QEMU user-mode (Linux CI with qemu-user)
```

`check-symbols` requires no emulator and validates that every arch compiles and exports the correct API.

13. How to Add a Primitive / Module / Keyword

Add a primitive (template = `libprim`): 1. Declare it in `libprim/include/prim.h` (the contract). 2. Implement it in **all five** `libprim/src/<arch>/prim.s` files, following that ISA's calling convention; use `CDECL(name)` for the symbol. 3. Add a check to `libprim/tests/test_prim.c`. 4. Validate: `make test (host)` · `make qtest-all (cross)` · `make check-symbols`.

Add a domain keyword: 1. Reserve it in `assembly/sakum_keywords.s` (machine registry) and mirror in `spec/spec_keywords.sak`. 2. Document the canonical Hinglish spelling in `SAKUM_HINGLISH.md`. 3. Implement the machine-level routine in `assembly/sakum_lib_domains.s` (+ ports).

Add a capability module (`.sakm`): declare a `capability.def` entry, implement the source, build to `.sakm` , sign with Ed25519, bundle into the static binary.

14. Roadmap

Milestone	Target
M1: Core runtime + module format + loader	—
M2: Capability registry + dependency graph + scheduler	—
M3: Worker pool + health monitor + healer	—
M4: macOS x86_64/ARM64 + Linux x86_64/ARM64	✓ partial (libprim)
M5: PDF + HTTP + TLS modules integrated	—
M6: Windows + RISC-V + Bare Metal	✓ riscv64 asm (partial)
M7: Formal verification + production hardening	—

15. Appendix A: Full keyword quick-reference

Control: aarambh naam kriya yadi anyatha yavat paryantam vapsa satya asatya shunya lek aur athava pariksha

Engine: hriday spand naadi sutra manas buddhi chetana prana atma

Data: vektor ank kech vektor anukra sthit asthit grantha sanchay varg sarani chakra smriti aavantan mukti

Systems: tantra anvesh paripath dvar map brahma jaal sampark

AI/Quantum: prajna buddhi chintan smaran adhigam anu adhisthiti samyojan

Security: raksha gopan vigopan praman mudra kunji kavach

Advanced: vyakhya svadhyaya santripti

(See §3.2 for the full 148-keyword registry by subsystem.)

16. Appendix B: Glossary

Term	Meaning
Sakum	The language: "together / with intent"
Hinglish	Romanized Sanskrit keywords (the language surface)
PAL	Platform Abstraction Layer — unified syscall surface
CT	Capability Token — signed permission/identity record
.sakm	Sakum module binary format
SIR	Sanskrit Intermediate Representation (portable hub)
libprim	Cross-platform machine-code primitive library (foundation layer)
CDECL	Macro applying the correct OS symbol prefix (_ on macOS)
#what	Binary-hash address of a construct (no SHA-256)
sutra	Creator-owned encryption key (installed, never printed)
hriday / spand / naadi	Heart (allocator) / Pulse (tick) / Nerve (bus)
sanchay	Unified six-shape database engine
brahma	Web crawler / learner activity
vektor	First-class SIMD vector
W^X	Writable-or-executable memory policy (security)
QCB1	Quantum Circuit Binary v1 format
RVV / NEON / AVX2	SIMD ISA extensions (RISC-V / ARM / x86)
QEMU user-mode	qemu-<arch> binaries that run a foreign-arch ELF on the host

End of Bible v1. Every subsystem in the repository is covered: doctrine, keywords, file types, language tour, query engine, compiler pipeline, cross-platform machine layer (platform.inc + libprim), runtime/capability model, self engine, build/test, and extension guide.